arXiv:2605.31002v1 [cs.DB] 29 May 2026

Modeling and Optimization for Massive Data Allocation in Database

Panpan Niu[1], Boxiang Ren[2], Hao Wu[1], Xin Yao[2]

[1]Department of Mathematical Sciences, Tsinghua University
[2]2012 Labs, Huawei Technologies Co., Ltd

June 1, 2026

Abstract

In the era of big data, e-commerce and Internet platforms face the challenge of processing massive amounts of data. However, due to data being scattered across different machines in distributed database, extra communication costs are incurred in gathering relevant data to complete transactions. Without a carefully designed data placement scheme, this cost can severely impact the performance of Online Transaction Processing systems. To meet industry requirements, algorithms that output a data placement scheme that achieves i) data balance and ii) low communication overhead within a fixed period of time are eagerly investigated. Although some existing methods have been studied, they do not adequately meet the aforementioned requirements. In this paper, inspired by the normalized cut of spectral clustering, we introduce a novel model for data allocation problem. The normalized cut reconciles the inherent conflict between the two objectives. Taking into account the variable characteristics of the model, we formulate the problem as a 0-1 optimization problem, and solve the relaxed problem using the Bregman proximal gradient method with guaranteed convergence. The numerical experiments reveal that the convergent solutions can be smoothly rounded to discrete solutions. Furthermore, our algorithm surpasses both simple and meta-heuristic partitioning schemes by minimizing migration costs while maintaining a superior balance.

1 Introduction

The database management system (DBMS) (Codd, 1970) has been applied in various fields, including e-commerce systems (Li, 2019; Ahmed et al., 2021), computerized library systems (Hernansyah et al., 2016) and geographic information systems (GIS) (Schneider, 1997). As technology advances, enterprises face an exponential increase in data volume, often reaching billions of units. For example, Google, the largest search engine company, handles more than 85.5 billion new visits per month. Storing such vast amounts of data in memory using a single-node DBMS is no longer feasible. To address these challenges, a distributed database management system (DDBMS) (Özsu and Valduriez, 1999) has been proposed, leveraging multiple servers to manage large-scale data. Things become far more complicated when executing a transaction under the distributed setting. The DDBMS must coordinate these servers to aggregate the relevant data blocks into one node

The first two authors contributed equally to this work.
Corresponding author: Xin Yao (yao.xin1@huawei.com).

before executing that transaction. However, if the data placement scheme is poorly designed, this process could cause serious data traffic and lead to a cliff-like drop in throughput (Pavlo et al., 2012; Shibata et al., 2010). Hence, the importance of developing an effective data partitioning strategy cannot be overstated.

In the field of databases, formulating an effective data partitioning strategy is commonly known as the Data Allocation Problem (DAP). Numerous algorithms have been proposed to solve DAP, including workload-agnostic partitioning techniques (DeWitt and Gray, 1992), greedy or heuristic algorithms (Atrey et al., 2020; Taft et al., 2014; Pavlo et al., 2012; Serafini et al., 2016), graph-based methods (Curino et al., 2010; Quamar et al., 2013; Golab et al., 2014), deep learning reinforcement algorithms (Hilprecht et al., 2019, 2020), etc. Among these, graph-based methods, such as those using METIS (Karypis, 1997) or hMETIS (Karypis and Kumar, 1998), two publicly available graph partitioning libraries, have shown great potential. However, the full potential of these methods in terms of partition performance remains underexplored (Golab et al., 2014). Furthermore, these algorithms inherently rely on various forms of greedy or heuristic techniques that focus on local graph properties, and a rigorous analysis of these approaches is often lacking (Khandekar et al., 2009; Desale et al., 2015; Stanton and Kliot, 2012). Therefore, there remains significant interest in developing methods that yield high-quality partitioning solutions with guaranteed convergence to solve DAP.

In recent years, researchers have formulated the DAP as a series of combinatorial optimization problems (Curino et al., 2010; Taft et al., 2014; Serafini et al., 2016; Quamar et al., 2013; Atrey et al., 2020; Golab et al., 2014; Yang et al., 2018; Firnkes, 2019). Notably, the Hypergraph Partitioning Problem and the Graph Partitioning Problem (GPP) are prominent representations of these approaches. In the case of the former (Quamar et al., 2013; Yang et al., 2018; Firnkes, 2019), which attempts to address a more general problem, historical workloads are modeled as a hypergraph, with each hyperedge corresponding to a transaction. However, hypergraph partitioning often results in worse performance compared to conventional graph partitioning (Golab et al., 2014). Therefore, this paper focuses on the GPP (Curino et al., 2010; Serafini et al., 2016). As noted in Pavlo et al. (2012); Curino et al. (2010), GPP aims to balance the workload across nodes while minimizing the number of transactions that require access to multiple servers. A key challenge in DAP is the inherent conflict between these two objectives (Serafini et al., 2016). Inspired by spectral clustering (Von Luxburg, 2007), we apply a graph-based Normalized Cut[1] (NCut) to address the challenges posed by GPP. The NCut model resolves these conflicting objectives through a unified function.

It has been proved by Shi and Malik (2000) that the problem of minimizing NCut is NP-Complete. Several approximation algorithms (Van Den Heuvel et al., 2008; Ng et al., 2001; Shi and Malik, 2000; Yan et al., 2009; Dhillon et al., 2004) have been proposed. For instance, Shi and Malik (2000) and Ng et al. (2001) successively propose two classical spectral methods. However, spectral methods are impractical in scenarios involving extensive databases due to their computational complexity of $O(N^3)$, with N the number of data points (Yan et al., 2009). A critical observation is the capacity of the Bregman Proximal Gradient (BPG) (Beck and Teboulle, 2009, 2003; Bauschke et al., 2003) method to address this challenge. Although BPG was initially proposed for convex optimization, it is also powerful when applied to constrained non-convex optimization problems with theoretical guarantees (Bolte et al., 2018).

In this paper, we introduce a novel model for the DAP by formulating it as an NCut model, in-

[1]Ratio Cut is also an available option. However, as demonstrated in (Nie et al., 2010), NCut often outperforms Ratio Cut and leads to more balanced clustering.

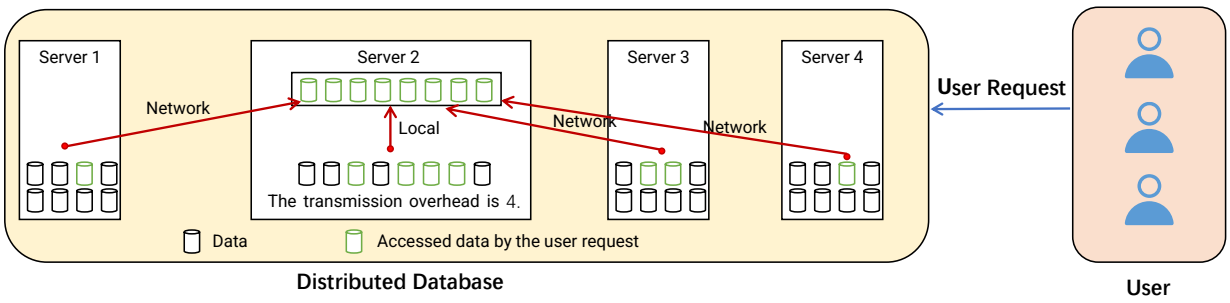


Figure 1: Data migration process in a distributed database. The process of data migration involves transferring data fragments between different nodes. The Data represents the actual data fragments stored in the leaf nodes of B+ trees (Elmasri and Navathe, 2016). They can be simply understood as blocks of data.

spired by spectral clustering. The sum-of-fractions structure of the NCut model effectively captures two major concerns of DAP: the numerators aim to reduce migration costs, while the denominators promote load balancing. To tackle the combinatorial nature of NCut, we reformulate it as an integer programming problem and further relax it into a continuous optimization problem. To address the resulting non-convexity of the relaxed problem, we employ the BPG method and provide a convergence analysis, ensuring that the algorithm reliably converges to high-quality solutions. Numerical experiments reveal that the convergent solutions obtained by BPG can be effectively rounded to discrete solutions, demonstrating the practical feasibility of our approach. Compared to simple and meta-heuristic partitioning schemes, our algorithm achieves superior performance by minimizing migration costs while maintaining better load balance. These results highlight the potential of our method to enhance data allocation strategies in distributed database systems, where high-quality solutions are crucial for improving system throughput.

This paper is organized as follows. Section 2 develops an NCut model for DAP. In Section 3, we use BPG method to solve the relaxed problem and provide proofs of convergence. The numerical experiments in Section 4 demonstrate the advantages of our approach. Finally, Section 5 concludes the paper.

2 DAP Modeling

We first outline the main components of distributed database systems. Figure 1 illustrates the processing flow of a transaction. The process begins when a user submits a transaction request, which can involve operations such as an update or a select query. The DDBMS selects a server to execute the procedural control code and the relevant queries (Pavlo et al., 2011, 2012). In practice, the selected server is typically the one that already stores the largest portion of the data required by the transaction. Unlike local transactions, distributed transactions involve data that is spread across multiple servers. In such cases, data residing on other servers must be migrated to the chosen server via network communication. For example, as shown in Figure 1, server 2 contains the majority of the relevant data, while the other servers transmit the remaining data blocks to it. Notably, the migration time involved in this process accounts for a large portion of the overall transaction processing time (Pavlo et al., 2012). The migration time primarily depends on the

number of transferred data. Specifically, migration time is typically orders of magnitude larger than the partitioning time. Therefore, we strategically allocate more time to the partitioning process to achieve a better partitioning result. This approach effectively reduces the overall migration overhead, thereby optimizing the system's overall performance. The main objective of the DAP in Online Transaction Processing (OLTP) systems is to minimize the number of distributed transactions while ensuring a balanced workload across all nodes. Interestingly, despite being formulated differently, the DAP shares significant similarities in optimization objectives and constraints with the Graph Partitioning Problem, as demonstrated by previous studies (Golab et al., 2014; Curino et al., 2010). Leveraging these similarities, we transform the DAP into a well-established GPP framework.

In the proposed model, the data blocks in the leaf nodes of B+ trees are treated as vertices. Let these vertices be represented as a set, denoted by $V = \{v_1, v_2, ..., v_N\}$. Co-accesses between two vertices are modeled as edges, with the weight defined by the number of transactions that simultaneously access both vertices. This results in a graph $G = (V, E)$, where E represents the set of edges. Minimizing the number of distributed transactions and maintaining a balanced workload across nodes are closely related to finding a balanced partitioning of the graph G (Curino et al., 2010). Specifically, given K subsets, a feasible partitioning scheme divides V into $P_1, P_2, ..., P_K$, such that $P_i \cap P_j = \emptyset$ for $i \neq j$, and $\cup_m P_m = V$. Let

$$\boldsymbol{X} = (x_{ij})_{N\times K} = (\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_K)$$

where

$$x_{ij} = \begin{cases} 1, & if \quad v_i \in P_j, \\ 0, & otherwise. \end{cases}$$

Since each data is uniquely assigned to a site, it satisfies the following no-replication constraint:

$$\sum_{i=1}^{K} \boldsymbol{x}_i = \mathbf{1}_N,$$

where $\mathbf{1}_N$ denotes the all-one vector in $\mathbb{R}^N$. Addressing the challenge of determining a suitable level of load imbalance constraint in GPP becomes challenging, as noted by Serafini et al. (2016). Here, we propose an NCut model that effectively tackles this challenge. First, we represent the two objectives of DAP in terms of equations involving $\boldsymbol{X}$. To quantify the workload of a component $P_i \subseteq V$, we define vol(P) as follows:

$$\text{vol}(P_i) := \sum_{v_j \in P_i} d_j = \sum_{v_s \in P_i, v_t \in V} w_{st} x_{si} = \boldsymbol{x}_i^T \boldsymbol{W} \mathbf{1}_N \tag{1}$$

where, $d_i = \sum_{j=1}^{n} w_{ij}$ represents the weighted degree of a node $v_i \in V$, and $\boldsymbol{W} \in \mathbb{R}^{N\times N}$ is the adjacency matrix of G. The d_i indicates the potential number of migrations for v_i across all transactions. Thus, vol(P_i), utilized later for balancing, denotes the communication load undertaken by server i. Next, we define cut of P_i as

$$\text{cut}(P_i) := \sum_{v_i \in P_i, v_j \notin P_i} w_{ij} = \sum_{v_s \in P_i, v_t \in V} w_{st} - \sum_{v_s \in P_i, v_t \in P_i} w_{st} = \boldsymbol{x}_i^T \boldsymbol{W} (\mathbf{1}_N - \boldsymbol{x}_i). \tag{2}$$

The cut(P_i) measures the total weight of edges connecting P_i with other partitions. From the perspective of DAP, the $\frac{1}{2}\sum_{i=1}^{K} cut(P_i)$ provides an approximation to the communication overhead induced by distributed transactions. Assuming each vertex has the same size and inspired

by Spectral Clustering (Von Luxburg, 2007), the objective and balanced constraint of GPP are approximately equivalent to minimizing the NCut:

$$\text{NCut}(P_1, .., P_K) = \frac{1}{2}\sum_{i=1}^{K}\frac{\text{cut}(P_i)}{\text{vol}(P_i)} = \frac{1}{2}\sum_{i=1}^{K}\frac{\boldsymbol{x}_i^T\boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T\boldsymbol{W}\mathbf{1}_N}.$$

On one hand, the function $\sum_{i=1}^{K}\frac{1}{\text{vol}(P_i)}$ achieves its minimum when all $\text{vol}(P_i)$ are equal. Thus, NCut assesses the load balancing of P_i within a scheme $\{P_1, P_2, ..., P_k\}$ using (1). On the other hand, NCut leverages (2) to evaluate the communication overhead among partitions. By combining these two components, the NCut model can serve as a unified evaluation criterion that simultaneously optimizes both objectives in DAP. This is empirically validated in Section 4 through numerical experiments.

For further elaboration, please refer to Shi and Malik (2000); Von Luxburg (2007).

In summary, the DAP is rewritten as the following optimization problem:

$$\mathcal{P}_1: \quad \min_{X} \quad \sum_{i=1}^{K}\frac{\boldsymbol{x}_i^T\boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T\boldsymbol{W}\mathbf{1}_N} \tag{3a}$$

$$s.t. \quad \boldsymbol{x}_i \in \{0,1\}^N, \quad \forall i \in \{1, 2, ..., K\} \tag{3b}$$

$$\sum_{i=1}^{K}\boldsymbol{x}_i = \mathbf{1}_N. \tag{3c}$$

This problem is NP-hard (Andreev and Räcke, 2004), which implies that computing exact solutions becomes computationally intractable for large-scale instances. Therefore, we focus on developing an efficient approximation algorithm that can provide near-optimal solutions within a reasonable computation time in the next section.

3 The Algorithm and Convergence Analysis

In this section, we present our approach for solving the relaxed problem, along with its theoretical convergence guarantees. Specifically, we develop a BPG-based approach for the relaxed problem of NCut problems and provide its convergence properties.

3.1 The Bregman Proximal Gradient algorithm for NCut

The problem $\mathcal{P}_1$ is an integer programming problem, which is difficult to tackle. To resolve it, taking into account (3c), we relax (3b) and obtain a continuous problem $\mathcal{P}_2$.

$$\mathcal{P}_2: \quad \min_{X} \quad \sum_{i=1}^{K}\frac{\boldsymbol{x}_i^T\boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T\boldsymbol{W}\mathbf{1}_N} \tag{4a}$$

$$s.t. \quad \boldsymbol{x}_i \geq 0, \quad \forall i \in \{1, 2, ..., K\} \tag{4b}$$

$$\sum_{i=1}^{K}\boldsymbol{x}_i = \mathbf{1}_N. \tag{4c}$$

Relaxations from integer programming problems to continuous problems often lead to challenges in the rounding process, i.e., transforming continuous solutions into discrete ones. Rounding techniques have been widely used and proven effective in many optimization scenarios, as demonstrated

in Karger et al. (1999); Buchbinder et al. (2017). Our numerical experiments further validate that our approach for the continuous problem yields solutions close to a 0-1 solution in practice, thereby facilitating the rounding process and ensuring practical applicability.

We now proceed to solve the relaxed problem $\mathcal{P}_2$. This is a non-convex optimization problem with linear constraints. Proximal gradient (PG) methods are well-suited for this class of problems, particularly those with row/column sum constraints, where explicit solutions are often attainable. By choosing Bregman divergence in the regularization term of PG, the non-negativity constraints are naturally absorbed into the objective function, resulting in a concise update rule. Moreover, existing theoretical analyses on the application of Bregman Proximal Gradient to non-convex and nonsmooth optimization can be leveraged to establish convergence guarantees for our problem. For optimization problems $\mathcal{P}_2$, we denote

$$\mathcal{C} = \left\{ \boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, .., \boldsymbol{x}_K\} \in R^{N\times K} \;\middle|\; x_{ij} \geq 0,\ \sum_{i=1}^{K} \boldsymbol{x}_i = \mathbf{1}_N \right\},$$

which is a product of probability simplexes, and

$$f(\boldsymbol{X}) = \sum_{i=1}^{K} \frac{\boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N}.$$

Considering the simplex constraints, we use the Bregman distance $D_h(\boldsymbol{x}, \boldsymbol{y}) := h(\boldsymbol{x}) - h(\boldsymbol{y}) - \langle \nabla h(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle$ (Bregman, 1967; Bauschke et al., 1997) with entropy function $h(\boldsymbol{x}) = \sum_{i,j} x_{ij} \ln x_{ij}$, which leads to a simpler iteration formula compared to Euclidean norm. Employing the BPG iteration scheme, we obtain the following iteration formula:

$$\boldsymbol{X}^{t+1} = \text{argmin}_{\boldsymbol{X}\in\mathcal{C}} \left\{ \sum_i \lambda_t \left\langle \frac{\partial f}{\partial \boldsymbol{x}_i}(X^t), \boldsymbol{x}_i \right\rangle + \sum_{i,j} x_{ij} ln \frac{x_{ij}}{x_{ij}^t} - (x_{ij} - x_{ij}^t) \right\}, \tag{5}$$

dc f x X + khong cách bregman(X, X_t)

where

$$\frac{\partial f}{\partial \boldsymbol{x}_i}(\boldsymbol{X}) = \frac{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N(\boldsymbol{W}\mathbf{1}_N - 2\boldsymbol{W}\boldsymbol{x}_i) - \boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)\boldsymbol{W}\mathbf{1}_N}{(\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N)^2}.$$

By utilizing the first-order optimality condition, the iterative formula can be written as follows:

Larrange:

$$\boldsymbol{X}^{t+1} = P_{\mathcal{C}}(\boldsymbol{X}^t \circ e^{-\lambda_t \frac{\partial f}{\partial \boldsymbol{X}}(X^t)}), \tag{6}$$

where $\circ$ denotes Hadamard product, and $P_{\mathcal{C}}(\cdot)$ represents the Bregman projection of $\mathcal{C}$. Since $\mathcal{C}$ represents a product of probability simplexes, we can obtain $P_{\mathcal{C}}(X)$ by dividing each row of X by the sum of the row.

Algorithm 1 shows the pseudo-code of our algorithm. Here we use superscript t to denote the t-th iteration of a variable and utilize fixed step size $\lambda_t = \lambda$ in (5) [2]. A constant ϵ is added to the denominator of NCut objective function in both numerical experiments and theoretical analysis for numerical stability.

Next, we analyze the computational complexity of Algorithm 1. The adjacency matrix $\boldsymbol{W}$ is represented by a compressed format, such as CSR (Compressed Sparse Row). It takes $O(KM)$

[2]The step size is a hyperparameter that requires careful tuning. A large value can cause divergence, while a minimal one may result in slow convergence (Zeiler, 2012). Fortunately, the algorithm demonstrates stability over a wide range of step sizes. More details are provided in Section 4.

Algorithm 1 Bregman Proximal Gradient algorithm for NCut

Require: Adjacency matrix of G $\boldsymbol{W} \in \mathbb{R}^{N\times N}$, number of partitions K, step size λ, and the number of iterations T

Ensure: Data partition $\boldsymbol{y} \in \{1, \cdots, K\}^N$

1: $\boldsymbol{X}^{(0)} \leftarrow \frac{1}{K} * \boldsymbol{J} + 0.1 * \boldsymbol{R}$, where $\boldsymbol{J} \in \mathbb{R}^{N\times K}$ and $\boldsymbol{R} \in \mathbb{R}^{N\times K}$ are all-one matrix and uniform random matrix in $[0, 1]$

2: Normalize $\boldsymbol{X^{(0)}}$ such that the sum of each row is 1

3: **for** $t = 0$ to $T - 1$ **do**

4: **for** $i = 1$ to K **do**

5: $\boldsymbol{g}_i^{(t)} \leftarrow \frac{(\boldsymbol{x}_i^{(t)})^T \boldsymbol{W}\mathbf{1}_N(\boldsymbol{W}\mathbf{1}_N - 2\boldsymbol{W}\boldsymbol{x}_i^{(t)}) - (\boldsymbol{x}_i^{(t)})^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i^{(t)})\boldsymbol{W}\mathbf{1}_N}{((\boldsymbol{x}_i^{(t)})^T \boldsymbol{W}\mathbf{1}_N)^2}$

6: $\boldsymbol{G}^{(t)} \leftarrow (\boldsymbol{g}_1^{(t)}, \boldsymbol{g}_2^{(t)}, \ldots, \boldsymbol{g}_K^{(t)})$

7: $\boldsymbol{Y}^{(t+1)} \leftarrow \boldsymbol{X}^{(t)} \circ \exp\left(-\lambda \boldsymbol{G}^{(t)}\right)$

8: Normalize $\boldsymbol{Y}^{(t+1)}$ row-wise to obtain $\boldsymbol{X}^{(t+1)}$

9: **for** $i = 1$ to N **do**

10: $y_i \leftarrow \text{argmax}_{1\le j\le K}\, x_{ij}^{(T)}$;

time to compute the expressions in line 5 because of matrix-vector multiplication, where M is the number of non-zero elements in the sparse matrix. And it takes $O(KN)$ time from line 7 to line 8. Notably, compared with the traditional Euclidean distance based Bregman iteration, our entropy distance approach eliminates the need for sorting operations and reduces the computational cost of the projection operator to $O(NK)$. Consequently, the overall computational complexity of the algorithm is $O(TKM)$.

3.2 Convergence Analysis

For the convenience of the proof, we first introduce the definition of L-smad here. More details of L-smad can be found in Bolte et al. (2018).

Definition 3.1. *(Bolte et al., 2018) A pair of functions (g, h) is L-smad if there exists $L \in \mathbb{R}_{++}$ such that $Lh - g$ is convex on $\mathcal{C}$.*

To employ the convergence analysis framework of BPG designed for non-convex and nonsmooth optimization problems, we need to establish the L-smad property of our relaxed problem, as stated in the following lemma.

Lemma 3.2. *Entropy function $h(\boldsymbol{X}) = \sum_{i,j} x_{ij} \ln x_{ij}$ and $g(\boldsymbol{X}) = \sum_{i=1}^{K} \frac{\boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon}$ satisfy L-smad in $\mathcal{C}$.*

Proof. The aim is to prove that there exists $L \in \mathbb{R}_{++}$ such that $Lh - g$ is convex on $\mathcal{C}$. By definition, we have

$$Lh(\boldsymbol{X}) - g(\boldsymbol{X}) = \sum_{i=1}^{K} \left[L \sum_{j=1}^{N} x_{ij} \ln x_{ij} - \frac{\boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon} \right].$$

Since $\boldsymbol{X}$ can be separated by columns, it suffices

$$\phi(\boldsymbol{X}) = L \sum_{j=1}^{N} x_{ij} \ln x_{ij} - \frac{\boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon}$$

is a convex function on $\mathcal{C}_1 = \{\boldsymbol{x} \in \mathbb{R}^n | \mathbf{0} \leq \boldsymbol{x} \leq \mathbf{1}_N\}$. For $h_1(\boldsymbol{X}) = L\sum_{i=1}^N x_i \ln x_i$, we have

$$\frac{\partial^2 h_1}{\partial x_i^2}(\boldsymbol{X}) = \frac{L}{x_i}$$
$$\frac{\partial^2 h_1}{\partial x_i \partial x_j}(\boldsymbol{X}) = 0.$$

Thus, the Hessian matrix of h_1 is diagonal on $\mathcal{C}_1$ with diagonal elements not less than L. For $g_1(\boldsymbol{X}) = \frac{\boldsymbol{x}_i^T \boldsymbol{W}(\mathbf{1}_N - \boldsymbol{x}_i)}{\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon}$, we have

$$\frac{\partial^2 g_1}{\partial x_i^2}(\boldsymbol{X}) = \frac{u_i(\boldsymbol{X})}{(\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon)^4},$$
$$\frac{\partial^2 g_1}{\partial x_i \partial x_j}(\boldsymbol{X}) = \frac{v_{ij}(\boldsymbol{X})}{(\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon)^4}.$$

Given that u and v are continuous functions and bounded on the compact set $\mathcal{C}_1$, there exists a constant $M > 0$ such that $|u_i(\boldsymbol{X})| \leq M$ and $|v_{ij}(\boldsymbol{X})| \leq M$ for all $i, j \in \{1, 2, ..., n\}$. Since $\boldsymbol{x}_i$ is a nonnegative vector, it follows that $(\boldsymbol{x}_i^T \boldsymbol{W}\mathbf{1}_N + \epsilon)^4 \geq \epsilon^4$. Therefore, each element of the Hessian of g_1 is bounded above by $\frac{M}{\epsilon^4}$. Finally, we set

$$L = \frac{NM}{\epsilon^4} + 1,$$

then the Hessian matrix of $\phi(\boldsymbol{X}) = h_1(\boldsymbol{X}) - g_1(\boldsymbol{X})$ is a symmetric strictly diagonally dominant matrix. Thus, $\phi(\boldsymbol{X})$ is a convex function on $\mathcal{C}_1$. The proof of Lemma 3.2 is completed. □

We have shown the L-smad property of (g, h) in our model. Next, we present our main convergence result.

Theorem 3.3. *Let $\{\boldsymbol{X}^t\}$ be the iterative sequence generated by BPG, and $0 < \lambda L < 1$. Then the following conclusions hold*

(1) $\lambda g(\boldsymbol{X}^{t+1}) \leq \lambda g(\boldsymbol{X}^t) - (1 - \lambda L) D_h(\boldsymbol{X}^{t+1}, \boldsymbol{X}^t)$, *the sequence* $\{g(\boldsymbol{X}^t)\}_{t\in\mathbb{N}}$ *is non-increasing;*

(2) $\sum_{t=1}^{\infty} D_h(\boldsymbol{X}^{t+1}, \boldsymbol{X}^t) < \infty$, *and* $D_h(\boldsymbol{X}^{t+1}, \boldsymbol{X}^t) \to 0$ *as* $t \to \infty$;

(3) $\min_{1\leq t\leq T} D_h\left(\boldsymbol{X}^t, \boldsymbol{X}^{t-1}\right) \leq \frac{\lambda}{T}\left(\frac{g(\boldsymbol{X}^0) - g_*}{1-\lambda L}\right)$, *where* $g_* = \inf_{\boldsymbol{X}\in\mathcal{C}} g(\boldsymbol{X}) > -\infty$.

Theorem 3.3 extends the proof methodology of Bolte et al. (2018) to our setting. It guarantees the non-increasing property of the objective function and provides convergence properties of the iteration sequence in terms of Bregman distance. Furthermore, these results can be extended to the Euclidean distance through the application of Pinsker's inequality.

Remark 1. *In this study, we have successfully demonstrated the convergence of our proposed iterative method. However, we have not provided a proof of convergence to the global optimum. This is primarily due to the fact that this problem is inherently NP-hard.*

4 Numerical Experiments

This section evaluates the effectiveness of the proposed Bregman Proximal Gradient algorithm through numerical simulations. Instead of relying on a specific distributed database system, we construct synthetic workloads that simulate real OLTP transactions. This setting allows us to focus on the algorithmic performance without interference from system-level factors such as I/O latency or network scheduling.

Each simulated workload is represented as an undirected weighted graph $G = (V, E)$, where each vertex corresponds to a data block and each edge weight indicates the number of transactions that jointly access two blocks. To generate G, we assume N distinct data blocks and synthesize N transactions. The transaction size follows a distribution $|T_i| \sim \lfloor \ln(N) + U(0, \log_{10} N) \rceil$, where $U(a, b)$ represents a uniform distribution over the interval $[a, b]$. Two data blocks are connected if they co-occur in the same transaction, and the edge weight w_{ij} equals the number of such co-occurrences. This procedure generates structured graphs that reflect the logical access patterns of OLTP transactions. The characteristics of all generated graphs are summarized in Table 1.

No. of Vertices	No. of Edges	Mean Weighted Degree
10,000	605,738	121.8894
20,000	1,216,335	122.0058
40,000	2,892,674	144.8925
80,000	6,794,955	170.0513
160,000	14,725,786	184.1762

Table 1: Characteristics of the Synthetic Graphs.

To mitigate the impact of randomness, all results are averaged over 10 experiments. In cases where the computational time exceeds 3600s, it is marked as "-". The BPG algorithm is implemented in Python 3.7, using the CuPy library to accelerate sparse matrix-vector multiplication on GPUs.

In this work, we compare BPG with the following baselines, including two heuristic algorithms and a clustering method.

- **Round-robin (RR)** (DeWitt and Gray, 1992) partitions data based on their data ID. Together with hash partitioning and range partitioning, they are simple algorithms widely adopted in the industry. Due to their similar performance, we only choose round-robin partitioning for comparison.
- **Spectral Clustering (SC)** (Von Luxburg, 2007) is a powerful clustering technique that leverages the eigenstructure of a similarity matrix to partition data into groups.
- **METIS** (Karypis, 1997) is the best-known software package in graph partition, widely used in the industry. Despite the availability of parallel versions such as ParMETIS (Karypis et al., 1997) and multi-threaded versions like mt-METIS (LaSalle and Karypis, 2013), we opted to use METIS as the comparative algorithm due to its superior partitioning performance.

4.1 Algorithm Verification

We first investigate the convergence behavior of BPG algorithm. Figure 2 illustrates the NCut convergence trajectories with respect to the number of iterations under different problem scales and

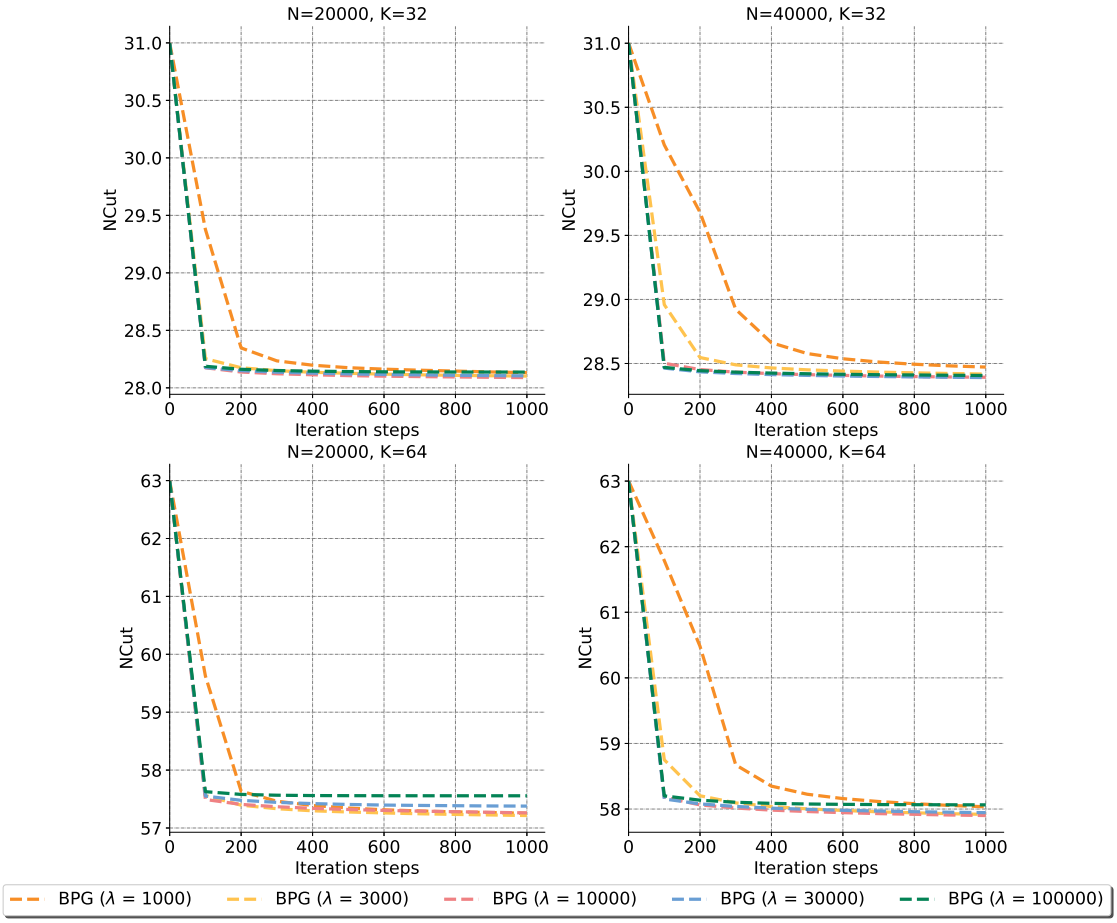


Figure 2: Convergence trajectories of NCut for the BPG algorithm with respect to the iteration steps under various step sizes (λ).

step sizes. Four synthetic DAPs (N, k) are considered:

$$(20000, 32), (20000, 64), (40000, 32), (40000, 64)$$

with the step sizes λ set to 1000, 3000, 10000, 30000, and 100000. As observed, all trajectories exhibit a monotonic decrease as iterations progress, and the empirical results strongly corroborate the theoretical convergence guarantee established in Theorem 3.3. From the figure, it shows that the BPG algorithm demonstrates remarkable stability across a wide range of step sizes, mitigating the risk of divergence or slow convergence discussed in Footnote 2. Consequently, we select $\lambda = 10000$ as the default step size for all subsequent experiments.

To further understand the effectiveness of the continuous relaxation, we analyze the properties of the converged solution matrix $\boldsymbol{X}$. Discrete problem $\mathcal{P}_1$ is transformed into a continuous problem $\mathcal{P}_2$ as outlined in Section 3. Here, we conduct experiments on the properties of the converged solution $\boldsymbol{X}$ from the continuous problem $\mathcal{P}_2$ to demonstrate the effectiveness of the relaxation. Table 2 summarizes the value distribution for 12 synthetic DAPs. The results indicate that the vast majority of elements are close to either 0 or 1, with intermediate values $(0.01-0.99)$ constituting less than 0.16%. This demonstrates that the relaxation preserves near-discrete structure, ensuring that the $\boldsymbol{X}$ can be smoothly rounded to the final partition. Furthermore, the number of elements

K	N	Count			Percentage		
		<0.01	>0.99	others	<0.01	>0.99	others
32	1×10^4	3.10×10^5	9.98×10^3	4.40×10^1	96.87%	3.12%	0.01%
	2×10^4	6.20×10^5	1.99×10^4	1.61×10^2	96.86%	3.11%	0.03%
	4×10^4	1.24×10^6	3.96×10^4	8.01×10^2	96.84%	3.09%	0.06%
	8×10^4	2.48×10^6	7.80×10^4	4.09×10^3	96.79%	3.05%	0.16%
64	2×10^4	1.26×10^6	1.99×10^4	1.25×10^2	98.43%	1.56%	0.01%
	4×10^4	2.52×10^6	3.97×10^4	5.94×10^2	98.43%	1.55%	0.02%
	8×10^4	5.04×10^6	7.86×10^4	2.84×10^3	98.41%	1.54%	0.06%
	1.6×10^5	1.01×10^7	1.53×10^5	1.46×10^4	98.36%	1.49%	0.14%

Table 2: Distribution of element values in the converged solution matrix $\boldsymbol{X}$. Within this table, columns 3-4 indicate the number of elements in $\boldsymbol{X}$ that are less than 0.01 and greater than 0.99, respectively. Column 5 represents the count of remaining elements. Columns 6-8 display the corresponding percentages for the preceding three columns.

exceeding 0.99 closely matches N, confirming that $\boldsymbol{X}$ effectively gives partition assignments for all vertices.

4.2 Algorithm Comparison

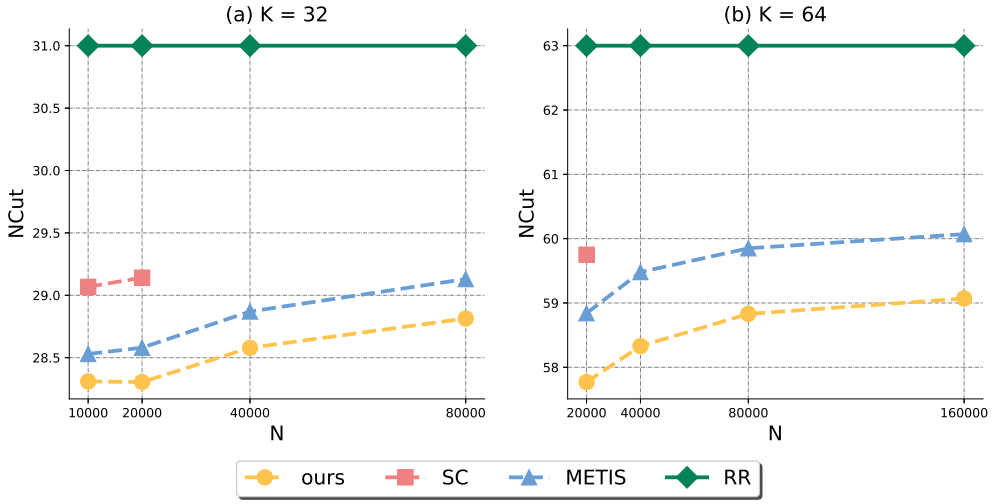


Figure 3: A comparison of BPG, SC, METIS and RR with data size N. Left: $K = 32$. Right: $K = 64$. The SC line is incomplete due to timeout issues.

After verifying convergence, we compare BPG against three baseline methods: SC, METIS, and RR. SC is executed using default parameters. For METIS, the ufactor parameter is set to 200 and 500 for $K = 32$ and $K = 64$, respectively. It's noteworthy that SC can be regarded as an alternative discrete approach for minimizing NCut. Due to the insights gained from the previous subsection, we set the number of iterations $T = 500$ for BPG to achieve a balance between solution quality and computational efficiency.

Figure 3 and Table 3 demonstrate that BPG consistently outperforms RR and SC in partition

K	N	NCut				Rate	Time (s)			
		ours	SC	METIS	RR		ours	SC	METIS	RR
32	1×10^4	28.31	29.07	28.53	31.00	0.78%	4.60	271.53	0.64	0.0009
	2×10^4	28.30	29.14	28.58	31.00	0.97%	5.08	1739.89	1.09	0.0019
	4×10^4	28.58	-	28.87	31.00	1.02%	5.15	-	2.08	0.0037
	8×10^4	28.81	-	29.13	31.00	1.09%	9.42	-	4.73	0.0072
64	2×10^4	57.77	59.75	58.84	63.00	1.86%	8.66	1802.72	1.52	0.0019
	4×10^4	58.33	-	59.48	63.00	1.96%	8.71	-	2.87	0.0036
	8×10^4	58.83	-	59.85	63.00	1.72%	17.90	-	6.17	0.0073
	1.6×10^5	59.07	-	60.07	63.00	1.69%	63.34	-	14.10	0.0154

Table 3: A comparison among the four algorithms for different synthetic DAPs (N, k). Columns 3-6 report the averaged NCut, Column 7 shows BPGs NCut reduction rate relative to METIS, and Columns 811 list average runtimes.

quality, achieving lower NCut values across all datasets. Compared with METIS, BPG attains slightly better partition quality, reducing NCut by approximately 1.4% on average. In terms of runtime, METIS remains the fastest method, benefiting from a highly optimized multilevel heuristic framework. Although BPG currently incurs higher computational cost than METIS, it operates substantially faster than the SC approach while maintaining a clear quality advantage. In practical database partitioning scenarios, such as initial sharding or periodic re-partitioning, this additional runtime is often acceptable, as improved partition quality directly translates into reduced communication overhead and better long-term system efficiency. Furthermore, since the present BPG implementation has not yet been fully optimized, significant potential remains to narrow the runtime gap in future work.

4.3 Performance Evaluation in DAP

According to Pavlo et al. (2012), it is stated that the number of data migration and the workload skew are two critical performance indicators in distributed databases. In order to provide an intuitive representation of the DAP, we adopt the Migration Cost (MCost) and the Mean Absolute Deviation (MAD) of the scheme as the evaluation metrics in this subsection. Consider a user-submitted task comprising L transactions $T = \{T_1, T_2, \ldots, T_L\}$, where $T_i \subset V$ for $i = 1, 2, \ldots, L$. Given a partitioning scheme $P = \{P_1, P_2, \ldots, P_K\}$, we define:

$$\begin{aligned} MCost(P, T) &= \sum_{i=1}^{L} \left(|T_i| - \max_{j=1,2,\ldots,K} (|T_i \cap P_j|) \right), \\ MAD(P) &= \frac{\sum_i \left| |P_i| - \frac{N}{K} \right|}{K}. \end{aligned} \tag{7}$$

Here, $MCost(P, T)$ measures the data migration overhead required to execute task T under partition P, while $MAD(P)$ quantifies workload imbalance across partitions.

The averaged MCost and MAD of the four algorithms on different data sizes with $K = 32$, and 64 are summarized in Table 4. A clear positive correlation emerges between the NCut values in Table 3 and the MCost and MAD results reported here. Lower NCut values generally correspond to reduced migration cost and improved load balance, indicating that NCut serves as a comprehensive

K	N	$MCost(P,T)$				$MAD(P)$			
		ours	SC	METIS	RR	ours	SC	METIS	RR
32	1×10^4	8.17×10^4	8.43×10^4	8.20×10^4	9.47×10^4	51.76	25.18	55.72	0.50
	2×10^4	1.64×10^5	1.69×10^5	1.64×10^5	1.89×10^5	76.26	36.76	111.50	0.00
	4×10^4	3.63×10^5	-	3.65×10^5	4.15×10^5	103.51	-	221.85	0.00
	8×10^4	7.98×10^5	-	8.05×10^5	9.03×10^5	141.18	-	442.68	0.00
64	2×10^4	1.70×10^5	1.76×10^5	1.71×10^5	1.97×10^5	116.75	43.45	124.05	0.50
	4×10^4	3.77×10^5	-	3.80×10^5	4.30×10^5	204.55	-	248.15	0.00
	8×10^4	8.31×10^5	-	8.36×10^5	9.34×10^5	323.85	-	495.11	0.00
	1.6×10^5	1.74×10^6	-	1.75×10^6	1.94×10^6	499.38	-	990.25	0.00

Table 4: Comparison of the four algorithms under different numbers of data (N). Columns 3-10 report the averaged MCost and MAD for each method.

indicator of partition quality, consistent with the formulation presented in Section 2. As shown in Table 4, BPG achieves consistently lower MCost than SC, METIS, and RR, with a reduction rate ranging from approximately 0.34% to 0.99% compared with the state-of-the-art METIS algorithm. Furthermore, BPG outperforms METIS in terms of MAD across most settings, indicating better workload balance. While SC and RR may achieve lower MAD values than BPG, SC suffers from high computational cost and RR yields substantially higher migration cost. Therefore, they are less competitive in the overall trade-off.

5 Conclusion

In this work, we investigated the data allocation problem within the context of database management systems. Our contributions are as follows. Firstly, we propose a novel model for tackling DAP. Second, taking into account the variable characteristics inherent in the model, we formulate the problem as a 0-1 optimization problem, and solve the relaxed problem using the BPG method with guaranteed convergence. Through comprehensive experiments, we show that our proposed BPG algorithm consistently outperforms existing methods in reducing data migration costs while maintaining superior workload balance.

References

Falah YH Ahmed, Reshma Sreejith, and Muhammad Irsyad Abdullah. Enhancement of e-commerce database system during the COVID-19 pandemic. In *2021 IEEE 11th IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE)*, pages 174–179, Penang, Malaysia, Mar. 2021. IEEE.

Konstantin Andreev and Harald Räcke. Balanced graph partitioning. In *Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures*, pages 120–124, Barcelona, Spain, 2004. ACM.

Ankita Atrey, Gregory Van Seghbroeck, Higinio Mora, Bruno Volckaert, and Filip De Turck. UnifyDR: A generic framework for unifying data and replica placement. *IEEE Access*, 8:216894–216910, 2020.

Heinz H Bauschke, Jonathan M Borwein, et al. Legendre functions and the method of random bregman projections. *Journal of Convex Analysis*, 4(1):27–67, 1997.

Heinz H Bauschke, Jonathan M Borwein, and Patrick L Combettes. Bregman monotone optimization algorithms. *SIAM Journal on Control and Optimization*, 42(2):596–636, 2003.

Amir Beck and Marc Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003.

Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.

Jérôme Bolte, Shoham Sabach, Marc Teboulle, and Yakov Vaisbourd. First order methods beyond convexity and lipschitz gradient continuity with applications to quadratic inverse problems. *SIAM Journal on Optimization*, 28(3):2131–2151, 2018.

Lev M Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

Niv Buchbinder, Roy Schwartz, and Baruch Weizman. Simplex transformations and the multiway cut problem. In *Proceedings of the 2017 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2400–2410. SIAM, 2017.

E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, Jan. 1970.

Carlo Curino, Evan Philip Charles Jones, Yang Zhang, and Samuel R Madden. Schism: A workload-driven approach to database replication and partitioning. *Proceedings of the VLDB Endowment*, 3(1–2):48–57, 2010.

Sachin Desale, Akhtar Rasool, Sushil Andhale, and Priti Rane. Heuristic and meta-heuristic algorithms and their relevance to the real world: a survey. *International Journal of Computer Engineering in Research Trends*, 2(5):296–304, 2015.

David DeWitt and Jim Gray. Parallel database systems: The future of high performance database systems. *Communications of the ACM*, 35(6):85–98, 1992.

Inderjit S Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means: spectral clustering and normalized cuts. In *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 551–556, Seattle, WA, USA, 2004. ACM.

Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of database systems*. Pearson Education, Boston, MA, USA, 7th edition, 2016.

Patrick Firnkes. Throughput optimization in a distributed database system via hypergraph partitioning. Master's thesis, Karlsruhe Institute of Technology, 2019.

Lukasz Golab, Marios Hadjieleftheriou, Howard Karloff, and Barna Saha. Distributed data placement to minimize communication costs via graph partitioning. In *Proceedings of the 26th International Conference on Scientific and Statistical Database Management*, pages 1–12, Aalborg, Denmark, 2014. ACM.

Herrnansyah, Yova Ruldeviyani, and Rizal Fathoni Aji. Enhancing query performance of library information systems using NoSQL DBMS: Case study on library information systems of Universitas Indonesia. In *2016 International Workshop on Big Data and Information Security (IWBIS)*, pages 41–46, Jakarta, Indonesia, 2016. IEEE.

Benjamin Hilprecht, Carsten Binnig, and Uwe Roehm. Learning a partitioning advisor with deep reinforcement learning, 2019. URL `https://arxiv.org/abs/1904.01279`.

Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. Learning a partitioning advisor for cloud databases. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 143–157, Virtual Conference, 2020. ACM.

David R Karger, Philip Klein, Cliff Stein, Mikkel Thorup, and Neal E Young. Rounding algorithms for a geometric embedding of minimum multiway cut. In *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*, pages 668–678, Atlanta, GA, USA, 1999. ACM.

George Karypis. Metis: Unstructured graph partitioning and sparse matrix ordering system. Technical report, Department of Computer Science, University of Minnesota, 1997.

George Karypis and Vipin Kumar. A hypergraph partitioning package. Technical report, Army HPC Research Center, Department of Computer Science and Engineering, University of Minnesota, 1998.

George Karypis, Kirk Schloegel, and Vipin Kumar. PARMETIS: Parallel graph partitioning and sparse matrix ordering library. Technical Report TR 97-060, Department of Computer Science and Engineering, University of Minnesota, Minneapolis, MN, USA, 1997.

Rohit Khandekar, Satish Rao, and Umesh Vazirani. Graph partitioning using single commodity flows. *Journal of the ACM (JACM)*, 56(4):1–15, 2009.

Dominique LaSalle and George Karypis. Multi-threaded graph partitioning. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 225–236, Boston, MA, USA, 2013. IEEE.

Feifei Li. Cloud-native database systems at Alibaba: Opportunities and challenges. *Proceedings of the VLDB Endowment*, 12(12):2263–2272, Aug. 2019.

Andrew Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems*, 14, 2001.

Feiping Nie, Chris Ding, Dijun Luo, and Heng Huang. Improved minmax cut graph clustering with nonnegative relaxation. In *Machine Learning and Knowledge Discovery in Databases*, pages 451–466, Berlin, Heidelberg, 2010. Springer.

M Tamer Özsu and Patrick Valduriez. *Principles of distributed database systems*. Springer, New York, NY, USA, 1999.

Andrew Pavlo, Evan PC Jones, and Stanley Zdonik. On predictive modeling for optimizing transaction execution in parallel OLTP systems. *Proceedings of the VLDB Endowment*, 5(2), 2011.

Andrew Pavlo, Carlo Curino, and Stanley Zdonik. Skew-aware automatic database partitioning in shared-nothing, parallel OLTP systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 61–72, Scottsdale, AZ, USA, 2012. ACM.

Abdul Quamar, K Ashwin Kumar, and Amol Deshpande. Sword: Scalable workload-aware data placement for transactional workloads. In *Proceedings of the 16th International Conference on Extending Database Technology*, pages 430–441, Genoa, Italy, 2013. ACM.

Markus Schneider. *Spatial data types for database systems: Finite resolution geometry for geographic information systems*. Springer, Berlin, Heidelberg, 1997.

Marco Serafini, Rebecca Taft, Aaron J Elmore, Andrew Pavlo, Ashraf Aboulnaga, and Michael Stonebraker. Clay: Fine-grained adaptive partitioning for general database schemas. *Proceedings of the VLDB Endowment*, 10(4):445–456, 2016.

Jianbo Shi and Jitendra Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.

Takeshi Shibata, SungJun Choi, and Kenjiro Taura. File-access patterns of data-intensive workflow applications and their implications to distributed filesystems. In *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, pages 746–755, Chicago, IL, USA, 2010. ACM.

Isabelle Stanton and Gabriel Kliot. Streaming graph partitioning for large distributed graphs. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1222–1230, Beijing, China, 2012. ACM.

Rebecca Taft, Essam Mansour, Marco Serafini, Jennie Duggan, Aaron J Elmore, Ashraf Aboulnaga, Andrew Pavlo, and Michael Stonebraker. E-store: Fine-grained elastic partitioning for distributed transaction processing systems. *Proceedings of the VLDB Endowment*, 8(3):245–256, 2014.

Martijn Van Den Heuvel, Rene Mandl, and Hilleke Hulshoff Pol. Normalized cut group clustering of resting-state fmri data. *PloS one*, 3(4):e2001, 2008.

Ulrike Von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.

Donghui Yan, Ling Huang, and Michael I Jordan. Fast approximate spectral clustering. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 907–916, Paris, France, 2009. ACM.

Wenyin Yang, Guojun Wang, Kim-Kwang Raymond Choo, and Shuhong Chen. Hepart: A balanced hypergraph partitioning algorithm for big data applications. *Future Generation Computer Systems*, 83:250–268, 2018.

Matthew D. Zeiler. Adadelta: An adaptive learning rate method, 2012. URL `https://arxiv.org/abs/1212.5701`.